# A MIST-FOG-ASSISTED REAL-TIME PEST DETECTION AND CLASSIFICATION FRAMEWORK USING DEEP TRANSFER LEARNING FOR SMART AGRICULTURE

A MAJOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT

FOR THE DEGREE OF

**MASTER OF SCIENCE IN COMPUTER SCIENCE**


SUBMITTED BY

**SUJEET KUMAR**

**Roll Number: 24234747005**

**Enrollment Number: 21ANDCBGCS000047**

**UNDER THE SUPERVISION OF**

**DR. DILIP SENAPATI**




**DEPARTMENT OF COMPUTER SCIENCE**

**FACULTY OF MATHEMATICAL SCIENCES**

**UNIVERSITY OF DELHI**

**DELHI - 110007, INDIA**


ACADEMIC YEAR 2025-2026

# Declaration

This is to certify that:

- The work contained in this project report is original and has been done by me under the supervision of my designated supervisor, Dr. Dilip Senapati.
- The work has not been submitted to any other Institute or University for any degree, diploma, or academic distinction.
- I have conformed to the norms, regulations, and guidelines prescribed in the Ethical Code of Conduct of the Department and University.
- Whenever I have used material (data, visual images, theoretical formulations, and code sequences) from external sources, I have given due credit and acknowledgment to them by appropriate citations in the text of the report and listed them in the references.

Date: May 29, 2026

Place: Delhi

**SUJEET KUMAR**

**Roll No: 24234747005**

**M.Sc. Computer Science**

**University of Delhi**

# Certificate

This is to certify that the project report titled "A Mist-Fog-Assisted Real-Time Pest Detection and Classification Framework Using Deep Transfer Learning for Smart Agriculture", submitted to the Department of Computer Science, University of Delhi, by Sujeet Kumar (Roll No. 24234747005), has been carried out under my direct supervision. This work is done in partial fulfillment of the requirements for the completion of the Master of Science (M.Sc.) in Computer Science.

It is further certified that this work is original, has been carried out under my guidance, and to the best of my knowledge, has not been submitted previously for the award of any other degree or diploma.

Date: May 29, 2026
Place: Delhi

**Dr. Dilip Senapati**
**Supervisor**
**Department of Computer Science**
**University of Delhi**
**Delhi - 110007, India**

# Acknowledgements

I take this opportunity to extend my sincere thanks to all those who helped me in undertaking this project and compiling this comprehensive report. I would like to express my profound gratitude to my supervisor, Dr. Dilip Senapati, Professor, Department of Computer Science, University of Delhi, for his invaluable guidance, continuous encouragement, constructive suggestions, and oversight throughout this project. It has been a privileged learning experience under his supervision.

I would also like to thank all the faculty members of the Department of Computer Science, University of Delhi, for their guidance, support, and for providing a highly stimulating academic environment during my M.Sc. curriculum. Their instructions and classes have laid the foundation for the theoretical and practical concepts utilized in this research.

I also take this opportunity to express my gratitude to my peers, friends, and lab-mates who shared valuable discussions and helped troubleshoot experimental configurations on embedded hardware setups. Their cooperation, constant support, and good wishes significantly enhanced my academic journey.

Finally, my deepest appreciation goes to my parents, Subhash Prasad Rajak and Savitri Devi, and my family members, for their unconditional love, continuous moral support, good wishes, and sacrifices, which have been instrumental in keeping me motivated and focused throughout the course of this work.

Date: May 29, 2026
Place: Delhi

**SUJEET KUMAR**
**Roll No: 24234747005**
**Department of Computer Science**
**University of Delhi**

# Abstract

The global agricultural sector faces unprecedented challenges driven by population growth and ecological volatility. Automated pest monitoring has emerged as a crucial approach to preserve crop yields and reduce broad-spectrum chemical pesticide use. However, traditional cloud architectures introduce severe communication delays, high bandwidth costs, and network dependencies that limit real-time field deployment. This project report presents a holistic distributed computing framework that operates natively at the farm boundary by leveraging a hierarchical Mist-Fog-Cloud topology. The proposed system deploys an explicitly optimized MobileNetV3-Large core directly onto localized edge workstations to achieve low-latency, autonomous inference. To handle the complex intra-class similarities and severe dataset skews inherent to agricultural tracking, a dynamic weighted random sampling routine is integrated into the data execution loop, balancing gradient descent updates across 9 distinct insect categories (aphids, armyworm, beetle, bollworm, grasshopper, mites, mosquito, sawfly, and stem borer).

Experimental evaluations on resource-constrained embedded hardware targets demonstrate that the framework achieves a high top-1 validation accuracy of 97.33% while requiring only 0.62 GFLOPs of computational complexity and a compact 16.4 megabyte storage footprint. By processing visual data packages within 23.5 milliseconds, the system delivers immediate localized diagnostics independent of external internet connectivity, establishing a highly generalizable strategy for smart farming infrastructure. Furthermore, a comparative benchmarking analysis demonstrates that our optimized MobileNetV3-Large core achieves the best operational balance, outperforming larger architectures like ResNet18, EfficientNet-B0, and EfficientNetV2-S in resource utilization, memory consumption, and inference latency, making it the most suitable model for direct physical integration into automated pheromone traps and mobile agricultural inspection robots.

**Keywords: Smart Agriculture 4.0, Deep Transfer Learning, Mist Computing, Edge Intelligence, Pest Detection, MobileNetV3-Large, Weighted Random Sampler.**

# Table of Contents

# List of Tables

# List of Figures

# List of Abbreviations

| | |
|---|---|
| AI | Artificial Intelligence |
| AMP | Automated Mixed Precision |
| API | Application Programming Interface |
| CAGR | Compound Annual Growth Rate |
| CNN | Convolutional Neural Network |
| CPU | Central Processing Unit |
| DL | Deep Learning |
| DUCS | Department of Computer Science, University of Delhi |
| FIFO | First-In-First-Out |
| FLOPs | Floating Point Operations |
| FPS | Frames Per Second |
| GFLOPs | Giga Floating Point Operations |
| GPU | Graphics Processing Unit |
| IDS | Intrusion Detection System |
| IoT | Internet of Things |
| LR | Learning Rate |
| MACs | Multiply-Accumulate Operations |
| mAP | mean Average Precision |
| ML | Machine Learning |
| QoI | Quality of Information |
| RAM | Random Access Memory |
| ReLU | Rectified Linear Unit |
| RGB | Red-Green-Blue |
| SBC | Single-Board Computer |
| TOC | Table of Contents |
| UC | University Course / Core |
| WAN | Wide Area Network |

# Chapter 1
# Introduction

## 1.1 Overview of Smart Agriculture 4.0 and Precision Farming

The global agricultural sector is currently undergoing a radical transformation characterized by the convergence of digital technologies, biological sciences, and autonomous systems. This fourth agricultural revolution, commonly referred to as Agriculture 4.0 or Smart Agriculture, represents a fundamental paradigm shift from traditional, experience-based farming practices to data-driven, precise intervention models. Under the pressure of a rapidly expanding global population, which is projected to reach nearly 10 billion by the year 2050, contemporary agronomy must dramatically increase production yields. However, this production expansion must be achieved under severe ecological constraints, including agricultural land depletion, water scarcity, and unpredictable climate patterns. To resolve these challenges, modern smart farming relies extensively on the Internet of Things (IoT) sensors, automated environmental telemetry, unmanned aerial vehicles, and localized data analytics to optimize resources, reduce operational inputs, and maximize harvest quality.

Precision farming operates on the core principle of spatial and temporal variation management. Rather than treating an entire farmland as a uniform entity, precision agriculture utilizes localized observations to apply inputs (such as water, fertilizers, and pest treatments) precisely where and when they are needed. The execution of this paradigm relies on a continuous loop of data collection, processing, and localized physical actuation. Embedded sensors collect multi-modal environmental information, including soil moisture levels, relative humidity, ambient temperature, and high-resolution visual feeds. This data is processed to generate actionable diagnostic models that guide localized spraying, automated irrigation, or targeted mechanical interventions. Ultimately, the successful deployment of Smart Agriculture 4.0 depends on establishing robust, autonomous computation pipelines that can operate directly within the extreme physical and electrical constraints of remote field environments.

## 1.2 Pest Threats in Crop Fields and Drawbacks of Traditional Methods

Among the major existential threats to global food security and crop yield stability, insect pest infestations represent a major source of structural degradation. According to reports from the Food and Agriculture Organization (FAO), plant pests and diseases are responsible for destroying between 20% to 40% of global crop yields annually, resulting in direct financial losses of hundreds of billions of dollars. Pests affect crops at every stage of growth, leading to defoliation, root damage, stem boring, and vascular decay, which drastically reduces the quantity and quality of harvests. For centuries, identifying pest clusters and managing insect outbreaks has relied almost exclusively on manual field inspection. Farmers walk the fields, visually scanning crops for damage signs or individual insects, and make decisions based on subjective experiences.

This manual monitoring strategy possesses severe drawbacks that render it highly ineffective in large-scale modern farming. First, manual inspections are incredibly labor-intensive, slow, and subjective, making regular, comprehensive monitoring of large farmlands physically impossible. Consequently, minor pest infestations are often overlooked in their early stages. By the time visual signs of damage become prominent enough to be noticed, the infestation has typically progressed to a point where localized control is no longer feasible. Second, this late detection routinely forces farmers to apply chemical pesticides uniformly across the entire farmland. The indiscriminate spraying of broad-spectrum pesticides has catastrophic ecological consequences. It accelerates chemical toxicity in soils, pollutes adjacent water tables through chemical runoff, and destroys beneficial insect populations (such as natural predators and pollinators), thereby disrupting local biodiversity.

Additionally, the continuous, uniform application of pesticides creates intense evolutionary pressure, causing target pest species to rapidly develop robust resistance to chemical treatments. This resistance forces farmers to use stronger, more toxic formulations, accelerating a destructive cycle of escalating chemical usage and environmental degradation. Furthermore, chemical pesticide residues on food products pose severe health risks to consumers. It is clear that transition to Smart Agriculture requires the development of automated, continuous visual monitoring systems that can detect and classify specific pest

species at their early, localized stages, enabling farmers to transition from farm-wide reactionary spraying to highly targeted, localized, and ecological interventions.

## 1.3 Edge Intelligence and Lightweight Deep Learning (Challenges and Solutions)

Recent progress in computer vision and deep learning has fundamentally changed automated visual monitoring and fine-grained classification. Deep convolutional neural networks (CNNs) have achieved outstanding performance in image classification, target localization, and anomaly detection, matching or even exceeding human precision in structured environments. In agricultural contexts, deep learning models can analyze high-resolution visual feeds to identify micro-anatomical structures (such as wing venation patterns, antenna lengths, and torso shapes) of specific insect species, enabling precise multi-class classification. However, a significant gap remains between laboratory-scale convolutional breakthroughs and practical, real-world field deployment. Standard deep learning architectures (e.g., VGG, ResNet, DenseNet) are computationally massive, containing tens of millions of trainable parameters and requiring billions of floating-point operations (FLOPs) per single forward inference pass.

Executing these heavy deep learning models requires massive processing cores and high power draws, making them entirely dependent on the high-performance computing hardware found in remote cloud data centers. In a standard cloud-centric layout, visual data captured by remote cameras in the fields must be transmitted upstream to a centralized cloud server for processing. This reliance on central cloud processing introduces severe operational bottlenecks in active farming environments. Agricultural farmlands are inherently isolated and rural, presenting conditions where network connectivity is weak, highly latent, or completely absent. Sending high-resolution, uncompressed image feeds from hundreds of distributed cameras across a farm to a remote server creates massive bandwidth strains, frequent packet drops, and fatal communication delays. In active pest mitigation, where an insect population can multiply exponentially within hours, even a few hours of network delay can render centralized cloud feedback useless.

To overcome these network bottlenecks, the paradigm of Edge Intelligence (or localized AI) has emerged, proposing that deep learning models should run directly on local hardware nodes

deployed at the physical boundary of the farm. By executing model inference locally, Edge Intelligence eliminates the dependency on external network uplinks, slashes communication delays to milliseconds, reduces bandwidth usage, and ensures data privacy and security. However, deploying deep learning models to local edge hardware introduces a severe resource constraint challenge. Edge deployment targets, such as Raspberry Pi boards or local microcontrollers, possess extremely limited computational capacity, restricted volatile memory, and tight thermal limits. Running massive convolutional backbones on these systems causes severe memory faults, high latency (several seconds per image), and rapid battery depletion, leading to thermal throttling or system failure.

This resource bottleneck demands the development of lightweight deep learning models engineered specifically for edge execution. The optimization of these models involves finding the optimal balance between classification precision and computational footprint. Lightweight architectures utilize specialized design principles, such as depthwise separable convolutions, inverted residuals, linear bottlenecks, and hardware-aware neural architecture search, to compress model size and reduce computational complexity by orders of magnitude. By designing compact, highly optimized convolutional backbones, Edge Intelligence can successfully deliver real-time, localized classification on low-power devices, providing a viable pathway for autonomous field deployment.

## 1.4 Hierarchical Computing: Mist, Fog, and Cloud Computing Paradigms

Decentralized computing offers a viable structural remedy to the limitations of cloud-centric architectures through the hierarchical coordination of Mist, Fog, and Cloud layers. This multi-tiered paradigm distributes computational tasks, storage allocations, and network capacity across the entire spectrum from the physical collection point to the central data center, optimizing resource utilization at every level. The proposed three-tier topology is designed to support real-time agricultural monitoring by leveraging the unique capabilities of each layer, creating a resilient, self-healing network that remains functional even during total external communication failures.

The Mist computing layer represents the absolute edge boundary of the network, residing directly at the physical point of data ingestion. This tier consists of a dense network of ultra-

low-power, resource-constrained nodes, including small microcontroller units (MCUs), low-resolution camera modules (such as ESP32-CAM), and basic environmental sensors integrated directly into pheromone insect traps or mounted on lightweight mobile monitoring robots. Given their extremely limited battery capacities and minimal processing power, mist nodes do not execute complex machine learning algorithms. Instead, their operational role is strictly focused on data ingestion, basic noise filtering, region-of-interest cropping (to strip away redundant background pixels and minimize data payload), and immediate wireless transmission over short-range, low-power protocols (e.g., LoRaWAN, Zigbee, or localized Wi-Fi mesh). By keeping the processing requirements at this layer minimal, the system maximizes the battery lifespan of field-deployed devices.

The Fog computing layer acts as the localized computational workhorse, positioned on-site within the farm infrastructure (e.g., in a secure utility shed or central farm workstation). This tier consists of intermediate single-board computers (such as Raspberry Pi boards, NVIDIA Jetson modules, or localized edge servers) that possess substantial processing power, dedicated memory, and constant power supplies. The fog node aggregates the incoming visual feeds from all distributed mist traps, manages execution queues, and runs the core lightweight deep learning model to perform instantaneous classification. By localizing the inference loop, the fog layer delivers diagnostics within milliseconds and operates independently of external internet availability. The Cloud computing layer occupies the highest tier, serving as a global analytical engine. Rather than uploading raw image feeds, the fog node transmits periodic, highly compressed metadata logs containing localized infestation coordinates and pest distributions. The cloud aggregates this data to track regional pest migration vectors and execute heavy model retraining cycles if accuracy drift is observed, creating a comprehensive, closed-loop network.

## 1.5 Project Objectives and Contributions

The primary objective of this project is to design, implement, and validate a fully functional, decentralized Mist-Fog-Cloud computing framework that enables real-time, autonomous pest detection and classification directly at the agricultural field boundary. By tightly integrating edge-tailored hardware configurations with an optimized deep transfer learning backbone, this research aims to bridge the operational gap between resource-constrained edge hardware

boundaries and complex computer vision tasks. The project addresses the systemic challenges of network latency, bandwidth saturation, and dataset imbalances that have historically restricted the practical deployment of automated smart monitoring systems in remote agricultural zones.

The main contributions of this work are summarized as follows:

- Development of a scalable, three-tier Mist-Fog-Cloud computing layout optimized for isolated field environments, demonstrating how localized processing eliminates external communication delays and maintains full diagnostic autonomy during total network blackouts.
- Implementation of an edge-tailored transfer learning scheme using an optimized MobileNetV3-Large core that runs efficiently within strict thermal, memory, and processing boundaries of low-power single-board computers.
- Introduction of an adaptive data balancing pipeline using a weighted random sampler within the training execution loop to eliminate majority class bias and ensure high generalization accuracy across deeply skewed agricultural dataset distributions.
- Integration of automated mixed precision (AMP) configurations and early stopping mechanisms to accelerate optimization convergence, reduce training memory footprints, and avoid overfitting.
- Comprehensive empirical profiling and benchmarking of the finalized edge core against standard convolutional architectures (EfficientNet-B0, ResNet18, EfficientNetV2-S) on physical edge hardware targets, establishing a highly generalizable strategy for smart farming infrastructure.

## 1.6 Organization of the Report

This major project report is structured logically into six chapters to provide a comprehensive, detailed overview of the research. The current chapter (Chapter 1) introduces the context of Smart Agriculture 4.0, highlights the threat of pest infestations, details the challenges of edge deployment, defines the Mist-Fog-Cloud hierarchy, and outlines the primary objectives and contributions of this project. The remainder of the report is organized as follows:

Chapter 2 evaluates historical and contemporary literature regarding distributed computing architectures in agriculture and lightweight deep learning models, identifying key research gaps that this project aims to resolve. Chapter 3 establishes the theoretical mathematical modeling approach, detailing network queuing formulations, hardware computational

complexity bounds, and the class-balanced optimization framework. Chapter 4 defines the primary methodology and implementation details, highlighting dataset preparation, data augmentations, the internal layers of the MobileNetV3-Large core, and training hyperparameters. Chapter 5 details the experimental findings, presenting a comprehensive analysis of training trajectories, confusion matrices, classification heatmaps, boxplot dispersions, latency histograms, and comparative benchmarking results. Finally, Chapter 6 concludes the report with a summary of findings and proposes future research directions.

# Chapter 2

# Related Work

## 2.1 IoT Architectures and Data Management in Smart Farming

The physical realization of decentralized computing frameworks in agriculture has been explored extensively in recent years, driven by the limitations of traditional cloud-centric IoT designs. In remote agricultural zones, the reliance on continuous cloud processing is severely constrained by unstable network connectivity, high latency, and high bandwidth costs. To circumvent these bottlenecks, a systematic survey by Kalyani and Collier [1] comprehensively details the necessary shift toward a combined Cloud-Fog-Edge computing architecture. They demonstrate that offloading time-sensitive tasks, such as real-time environmental monitoring and anomaly detection, to edge and fog layers drastically reduces network latency, minimizes energy consumption on battery-powered sensors, and preserves the cloud layer for long-term, heavy-duty historical analytics.

The implementation of such hierarchical layers has been investigated through various specialized agricultural frameworks. For instance, Gómez et al. [2] presented FARMIT, a multi-tiered platform designed for the continuous assessment of crop quality in smart farming. The FARMIT architecture is structured across three distinct planes: Physical, Edge, and Cloud. The edge plane is directly responsible for managing localized crop monitoring devices and aggregating heterogeneous sensor data, including both visual RGB feeds and environmental metrics. By aggregating and filtering data locally, the edge plane effectively reduces the volume of raw data transmitted upstream, preventing network congestion. However, pushing computation to physically exposed farm areas introduces new challenges regarding data integrity and system security. Javeed et al. [3] address this by proposing an intrusion detection system (IDS) tailored for edge-envisioned smart agriculture operating in extreme environments. They argue that edge nodes are highly susceptible to physical tampering and cyber attacks, and develop a localized IDS that operates autonomously without relying on constant cloud connectivity, underscoring the critical need for complete edge autonomy.

Furthermore, the sheer volume of data generated by multi-modal agricultural sensors necessitates rigorous localized data management to avoid bandwidth saturation. Abdalzaher et al. [4] investigate the concept of Quality-Focused IoT data management, highlighting that transmitting all gathered raw data to the cloud is neither feasible nor desirable. They propose that edge nodes must act as intelligent filters that clean, annotate, and assess the Quality of Information (QoI) of high-velocity data streams. By dropping redundant, noisy, or low-quality visual feeds at the edge, the overall architecture avoids network saturation and ensures that upper fog layers only process high-value diagnostic information. These works collectively establish the architectural feasibility of decentralized agricultural systems, yet they rarely detail the internal hyperparameter optimization or class-balancing techniques required to run complex, multi-class visual models on these edge layers.

## 2.2 Lightweight Deep Learning models for Pest & Crop Monitoring

While architectural advancements provide the necessary communication and computing infrastructure, deep learning (DL) models serve as the cognitive engine for automated visual crop assessment. In precision agriculture, deep learning is primarily utilized for crop yield prediction, plant disease identification, and pest detection. However, standard convolutional neural network (CNN) architectures, such as ResNet, VGG, or DenseNet, contain tens of millions of parameters and require extensive processing cycles, making them computationally too heavy to run on the resource-constrained edge and fog hardware nodes deployed in the fields. To address this computational bottleneck, recent research has aggressively pivoted toward lightweight, mobile-friendly deep learning models that balance classification precision against a minimal footprint.

A highly relevant study by Li et al. [5] tackled the specific issue of real-time pest detection against complex agricultural backgrounds. Recognizing the heavy parameter and FLOP burden of traditional models, they proposed a lightweight locality-aware model that replaced standard heavy backbones with a compact MobileNetV3 architecture integrated with a YOLO detector. Their methodology proved that by utilizing MobileNetV3, they could drastically reduce the model's parameters and GFLOPs while maintaining, and in some cases improving, the mean Average Precision (mAP) for small pest targets. This demonstrates that specialized, lightweight architectures are fully capable of handling fine-grained visual classification tasks

directly on constrained edge devices. Moreover, Gómez et al. [2] suggested that predictive models are significantly more robust when visual pest detection is contextually supported by surrounding environmental data gathered at the edge. These findings highlight the potential of lightweight deep learning when integrated natively into localized edge gateways, but there remains a critical gap in analyzing how these compressed models handle the severe class imbalances and dynamic queuing delays inherent to active physical deployments.

## 2.3 Synthesis and Identification of the Research Gap

The review of existing literature establishes a clear consensus: the future of sustainable, automated precision agriculture relies on decentralized Cloud-Fog-Edge architectures that feature localized data filtering and secure edge operations, powered by highly compressed, lightweight deep learning models like MobileNetV3 for continuous crop and pest assessment. Despite these individual advancements, a notable research gap remains at the intersection of these domains, particularly when transitioning from controlled laboratory simulations to active physical deployments. First, lightweight pest detection studies frequently treat the deployment hardware as an afterthought, ignoring the dynamic queuing delays, packet loss, and network transmission overheads that occur when multiple mist node cameras flood a shared fog workstation simultaneously. Second, architectural IoT studies outline robust data pathways but rarely detail the internal transfer learning strategies, class-balancing techniques, or hyperparameter optimizations required to make a complex, multi-class visual model function accurately within those network and hardware constraints.

In this project, we bridge this divide by proposing a holistic framework that tightly couples a three-tier Mist-Fog-Cloud architectural hierarchy with a meticulously optimized, class-balanced MobileNetV3-Large core. By integrating robust data augmentation, automated mixed precision, and early stopping mechanisms natively designed for edge execution, our approach not only achieves high classification accuracy across skewed insect profiles but actively manages the computational, queuing, and memory realities of real-time agricultural IoT environments. This research provides a complete, verified solution that ensures edge intelligence can be reliably deployed to protect crop yields without relying on external cloud connectivity.

Chapter 3

System Architecture and Performance Modeling

## 3.1 Proposed Three-Tier Mist-Fog-Cloud Topology

To establish a reliable, low-latency, and autonomous computational framework, this project proposes a decentralized three-tier Mist-Fog-Cloud computing topology. Farmlands are often located in remote regions with highly unstable wireless network coverage, making centralized cloud-based systems impractical. The proposed hierarchical architecture overcomes this limitation by localizing the heavy visual classification tasks at the farm boundary, utilizing three distinct functional layers: the Mist layer, the Fog layer, and the Cloud layer. This distributed design optimizes bandwidth consumption, minimizes communication latency, and ensures complete operational autonomy even during total external network failures.

Figure 1 illustrates the overall system architecture and the flow of visual data packages through the three-tier network. At the lowest level, the Mist layer consists of localized sensor traps equipped with high-resolution camera modules that capture raw visual feeds of insect targets. These nodes apply a localized region-of-interest crop to strip away background pixels and compress the image data before transmitting it over a low-power, short-range wireless channel (such as LoRaWAN or local Wi-Fi) to the Fog layer. The Fog layer, consisting of a localized workstation, runs the core MobileNetV3-Large model to perform real-time classification. Finally, the Cloud layer receives only periodic, compressed metadata updates for global tracking and macro-level analysis, eliminating the need to transmit raw images upstream.

## 3.2 Hierarchical Latency and Network Transmission Modeling

To formalize the network behavior of the proposed topology, we model the system latency using a multi-tier queuing framework. Let M = {m_1, m_2, ..., m_K} represent the set of K active mist camera nodes deployed across the farmland, where each node captures and transmits images at an average Poisson arrival rate of lambda_k frames per second. The transmission of a visual data packet containing a single cropped frame from a specific mist node m_k to the shared fog workstation incurs a communication overhead, denoted as

T_net(k). This network delay is a function of the data payload size S_k in bits and the instantaneous channel capacity C_k in bits per second, which is formally defined as:

$$\boldsymbol{T_{net}(k) = \frac{S_k}{C_k} + \delta_{prop}} \qquad \textbf{(3.1)}$$

where delta_prop represents the physical propagation delay of the localized wireless channel. Because the central fog node acts as a single server processing streams from multiple mist units, the incoming data packets enter a local processing buffer. This incoming stream conforms to an M/M/1 queuing model where the cumulative arrival rate Lambda entering the fog gateway is the summation of all localized streams:

$$\boldsymbol{\lambda = \sum_{k=1}^{K} \lambda_k} \qquad \textbf{(3.2)}$$

To ensure complete framework stability, the local processing rate of the fog workstation node, denoted as mu_fog, must strictly exceed the cumulative arrival rate, bounding the system utilization factor rho as:

$$\boldsymbol{\rho = \frac{\lambda}{\mu_{fog}} < 1} \qquad \textbf{(3.3)}$$

The total expected waiting and queuing time W_queue experienced by any incoming image frame within the fog buffer before inference can begin is modeled using the Pollaczek-Khinchine formula:

$$\boldsymbol{W_{queue} = \frac{\lambda}{2\mu_{fog}(\mu_{fog} - \lambda)}} \qquad \textbf{(3.4)}$$

By distributing tasks based on these queuing metrics, the framework actively prevents the buffer overflows and packet drops that occur in standard cloud-centric systems during high-intensity pest swarms.

## 3.3 Computational Execution Bounds on Constrained Embedded Hardware

Once an image frame is successfully cleared from the arrival queue, it undergoes immediate deep learning classification via the optimized MobileNetV3-Large backbone. The localized computational performance is bounded by the hardware constraints of the embedded fog unit.

The total execution latency T_exec required to process a single forward pass of the neural network is mathematically modeled as a function of the total floating-point operations (FLOPs) and the computational capacity of the edge hardware.

Let Phi represent the total number of Multiply-Accumulate (MAC) operations within the MobileNetV3-Large architecture, and let Gamma denote the peak processing throughput of the localized edge accelerator or CPU expressed in floating-point operations per second (FLOPS). The execution latency is bounded by the structural layer configuration:

$$\boldsymbol{T_{exec} \geq \frac{2\Phi}{\Gamma \eta_{hardware}} + \Delta_{memory}} \qquad \textbf{(3.5)}$$

where eta_hardware represents the empirical hardware efficiency coefficient (eta_hardware in (0, 1]), which accounts for hardware-specific runtime constraints such as memory bandwidth limitations, thread synchronization overheads, and thermal throttling. The term Delta_memory encapsulations the explicit data serialization and structural copy latencies incurred when transferring image tensors from the localized buffer memory into the active processing caches of the execution core.

To achieve low-latency inference on resource-constrained edge units, the model compression pipeline replaces standard heavy convolutional operations with depthwise separable convolutions. The computational cost of a standard convolutional layer operating on an input tensor of size H x W x D_in with a kernel size D_K and D_out output channels is compared against the depthwise separable implementation. The computational reduction factor sigma_comp achieved by our approach is defined as:

$$\boldsymbol{\sigma_{comp} = \frac{D_K^2 \cdot D_{in} \cdot H \cdot W + D_{in} \cdot D_{out} \cdot H \cdot W}{D_K^2 \cdot D_{in} \cdot D_{out} \cdot H \cdot W} = \frac{1}{D_{out}} + \frac{1}{D_K^2}} \qquad \textbf{(3.6)}$$

Given that modern deep learning layers utilize high output channel densities where D_out >> 1, this structural formulation mathematically guarantees a computational reduction of approximately D_K^2 times, reducing the parameter burden to fit within tight embedded memory boundaries.

## 3.4 Class-Balanced Optimization and Information Maximization

Real-world insect tracking environments frequently encounter severely skewed distributions where common pests heavily outnumber rare, highly destructive species. Training a standard deep neural network under these conditions using a uniform categorical cross-entropy loss function introduces a severe majority class bias. This causes the network to maximize global validation scores by overfitting to dominant classes while failing to classify rare targets. To resolve this imbalance, our performance approach models a class-balanced information maximization framework.

Let N = {n_1, n_2, ..., n_C} represent the total number of training samples available across the C distinct pest classes in the dataset. The effective number of samples E_c for a specific target class c is modeled using a volume scaling parameter beta:

$$E_c = \frac{1 - \beta^{n_c}}{1 - \beta} \qquad (3.7)$$

where beta = (V - 1) / V is a hyperparameter controlled by the total volume of the feature space V. To counter majority dominance, a dynamic sampling weight w_c is calculated for each class and integrated into the dataloader via a weighted random sampler pipeline. The specific target weight is inversely proportional to its effective sample volume:

$$w_c = \left(\frac{1}{E_c}\right) \cdot \left(\sum_{j=1}^{C} \frac{1}{E_j}\right)^{-1} \qquad (3.8)$$

During the active training phase, the loss computation is dynamically scaled using these calculated weights. Let y denote the ground-truth label vector, and let y_hat represent the predicted softmax probability distribution output by the MobileNetV3-Large core. The class-balanced objective function L_CB minimized during training is formulated as:

$$L_{CB}(y, \hat{y}) = -\sum_{c=1}^{C} w_c \cdot y_c \cdot \log(\hat{y}_c) \qquad (3.9)$$

By optimizing this class-balanced loss function rather than a standard uniform cross-entropy metric, the gradient updates are scaled to prioritize minority class errors, ensuring that fine-grained structural variations among rare insect targets are actively preserved during backpropagation.

# Chapter 4
# Methodology and Implementation

## 4.1 Dataset Description and Statistics

The empirical validity of fine-grained visual classification depends heavily on the quality, diversity, and representing coverage of the underlying training data. This framework utilizes a multi-class agricultural pest dataset containing 9 distinct insect categories (aphids, armyworm, beetle, bollworm, grasshopper, mites, mosquito, sawfly, and stem borer), reflecting the visual complexities and high intra-class similarities common to real-world smart farming scenarios. Visual inspection of these classes reveals major challenges: species like aphids and mites are extremely small, requiring high-resolution details to resolve, while others like beetle and sawfly present high visual overlaps, sharing segmented torso layouts, similar coloration, and wing shapes. The dataset is split into an 80:20 training and validation distribution to ensure robust evaluation bounds.

To investigate the dataset's numerical characteristics, the raw sample frequency was compiled. The dataset presents a severe class imbalance. Common pests (such as aphids and beetle) possess high instance counts, representing the majority classes, whereas highly destructive but less frequent species (such as sawfly and stem borer) are significantly underrepresented. In a standard uniform training loop, this raw skew causes backpropagation to bias model parameters toward the dominant classes, leading to catastrophic misclassification rates for minority targets. This distribution imbalance justifies the integration of the dynamic weighted random sampler and class-balanced loss function within our methodology.

## 4.2 Spatial Transformations and Augmentation Pipeline

To prevent severe overfitting caused by the uniform background conditions of static insect traps and varying outdoor lighting, a dynamic spatial augmentation pipeline is implemented natively within the training dataloader. The pipeline introduces geometric and environmental variations on the fly, forcing the neural network to focus exclusively on fine-grained anatomical pest characteristics rather than memorizing localized ambient artifacts. The visual feeds captured by field cameras are first subjected to spatial standardization, where all

incoming image tensors are dynamically resized to a uniform dimension of 224 x 224 pixels. This size is selected to align precisely with the input requirements of the optimized deep transfer learning backbone, ensuring constant memory allocation during the forward pass.

Following resizing, raw pixel values ranging from 0 to 255 are normalized into floating-point representations within a standardized [0, 1] continuum, and subsequently scaled using global mean values mu = [0.485, 0.456, 0.406] and standard deviations sigma = [0.229, 0.224, 0.225] derived from ImageNet distributions. This scaling accelerates gradient descent stabilization during the fine-tuning stage. The dynamic augmentations applied to the training distribution during each epoch include:

- Random horizontal and vertical reflections with an execution probability of p = 0.5 to simulate arbitrary insect positioning within the trap orientation.
- Random affine rotations restricted within a range of [-30 degrees, +30 degrees] to counter variations in directional camera mounting angles.
- Dynamic color jittering with a maximum variation factor of 0.2 across brightness, contrast, saturation, and hue to mimic shifting ambient light conditions in open fields across day and night cycles.
- Random perspective transformations with a scale of 0.2 to train the network to maintain high classification accuracy when tracking distorted or partially occluded insect targets.

By ensuring that the training loop encounters unique variations of the insect profiles during each epoch, the augmentation pipeline significantly improves the out-of-domain generalization of the model when deployed onto new, unmonitored farm sectors.

## 4.3 Deep Transfer Learning via Optimized MobileNetV3-Large

The core classification capability of the fog workstation is powered by an explicitly optimized MobileNetV3-Large neural network backbone, pre-trained on ImageNet. The architectural choices are guided by the critical requirement to sustain high validation accuracy while operating within strict embedded memory limits and low thermal envelopes. The structural framework replaces heavy standard convolutional blocks with an optimized pipeline built around depthwise separable convolutions, inverted residual connections, linear bottlenecks, and localized coordinate attention mechanisms, reducing computational complexity by an order of magnitude.

The fundamental building block of the network is the depthwise separable convolution layer. In standard convolutional configurations, a single layer simultaneously alters both spatial dimensions and channel depths, requiring an enormous parameter footprint. Our framework decouples this process into two separate steps. First, a depthwise convolution applies a single, isolated spatial filter to each individual input channel. Second, a pointwise convolution utilizing a 1 x 1 kernel calculates a linear combination across the channel dimensions to generate new feature maps. This structural split drastically slashes the mathematical complexity of the forward pass, lowering the overall GFLOP burden by an order of magnitude without compromising the structural feature resolution.

To maximize information preservation as activations traverse deep layer hierarchies, the architecture implements inverted residual blocks with linear bottlenecks. Unlike standard residual networks that compress feature spaces internally, the inverted bottleneck first projects the incoming low-dimensional tensor into a significantly higher-dimensional space using a 1 x 1 expansion convolution. Spatial feature tracking occurs within this expanded zone. The layer concludes with a restrictive 1 x 1 projection convolution that returns the tensor to its original low-dimensional format. Crucially, the activation function is completely omitted from the final projection stage. Maintaining a linear output layer prevents non-linear rectifiers from destroying vital structural data when compressing high-dimensional feature maps down to tight channel bottlenecks.

The model further incorporates localized coordinate attention and squeeze-and-excitation modules to enhance fine-grained pest tracking. The squeeze-and-excitation pathway applies global average pooling across spatial grids to compress the channel distributions into a singular global descriptor vector. This vector passes through a tight bottleneck layer before being expanded back to the original channel dimensions, computing a set of dynamic, per-channel excitation weights. These weights scale the underlying feature maps to emphasize critical diagnostic areas, such as insect wing patterns or antenna structures, while actively suppressing irrelevant noise from the surrounding physical trap background.

Computational efficiency is further refined by substituting traditional, high-cost non-linear activations with the optimized hard-swish (h-swish) function. Standard swish curves require

calculating exponential sigmoid transformations, which introduce severe computational overhead on resource-constrained edge CPUs. The hard-swish function bypasses this bottleneck by approximating the smooth curve using a piece-wise linear formulation. This algebraic simplification allows the deployment hardware to execute non-linear activations using basic bitwise shifting and addition operations, preserving battery reserves and preventing thermal throttling on the deployed edge nodes.

## 4.4 Training Configurations and Hyperparameters

The final stage of the methodology establishes the precise configurations and optimization routines used to train the pest detection core. To completely eliminate majority class dominance, our framework integrates a dynamic data balancing architecture built directly into the PyTorch dataloading execution loop. During initialization, the total count of training samples across each of the 9 target classes is compiled into a global distribution dictionary. For every single image sample, an explicit sampling probability is computed that is inversely proportional to the frequency of its matching class label. These probabilities are loaded into a WeightedRandomSampler pipeline. During batch creation, the sampler dynamically draws images based on these computed weights. This operational pipeline ensures that every training batch presents a uniform, balanced distribution of classes, forcing the optimization algorithm to distribute gradient updates evenly across both common and minority insect categories, thereby avoiding majority class overfitting.

The network optimization uses the advanced AdamW optimizer, which introduces decoupled weight decay to prevent model parameters from expanding excessively during extended training sequences. The initial learning rate is anchored at alpha = 1 x 10^-3, paired with a decoupled weight decay parameter of 1 x 10^-4 and a rigorous cosine annealing learning rate scheduler that smoothly scales down the step size as training advances to ensure stable convergence in tricky local minima. To maximize hardware throughput and accelerate convergence, the training pipeline utilizes automated mixed precision (AMP) execution. This technique executes the standard forward and backward passes using fast 16-bit floating-point representations, while maintaining a master copy of the weights in high-precision 32-bit formats, reducing the local memory footprint by half and doubling the processing speed on modern computing hardware.

To prevent overtraining and ensure the framework remains highly generalizable, an automated early stopping mechanism monitors validation performance. If the computed validation loss fails to show a minimum structural improvement of 1 x 10^-4 over five consecutive epochs, the execution loop immediately terminates training and exports the optimal model parameters. The final model is serialized into a highly compressed format and loaded onto the local fog workstation, establishing an autonomous, real-time pest monitoring hub that protects crop yields without relying on external cloud infrastructure.

# Chapter 5
# Experimental Results and Discussions

## 5.1 Experimental Setup and Hardware Architecture

The empirical evaluations were executed on an infrastructure designed to match the proposed tiered topology layout. The training phase was conducted on a local workstation configuration equipped with an dedicated graphics accelerator possessing 16 gigabytes of video memory, running a Linux operating system environment with PyTorch framework bindings. To simulate the physical realities of field deployment, the optimized models were serialized and exported to a constrained edge target node representing the local farm fog layer gateway. The edge hardware node consists of a low-power single-board computer (Raspberry Pi 4 Model B with 4GB RAM) running Raspberry Pi OS. Visual feeds were simulated using test sets from the 9-class agricultural dataset, allowing a comprehensive critique of model accuracy, inference latency, memory footprints, and CPU utilization under realistic operational conditions.

## 5.2 Dataset Volumetric Distribution and Imbalance Profile

As discussed in Chapter 4, managing multi-class agricultural data requires understanding the underlying class distribution skew. Figure 2 visualizes the multi-class dataset volumetric distribution and target imbalances across the 9 categories. Analysis of this distribution profile reveals that dominant categories like aphids and beetle have high frequencies, whereas minority classes like sawfly possess extremely limited samples. In standard deep learning setups, this volumetric discrepancy would bias the gradient updates. The integration of the WeightedRandomSampler successfully resolved this imbalance, forcing the dataloader to balance representation dynamically and ensuring that the optimization loop distributed backpropagation weights evenly across all categories.

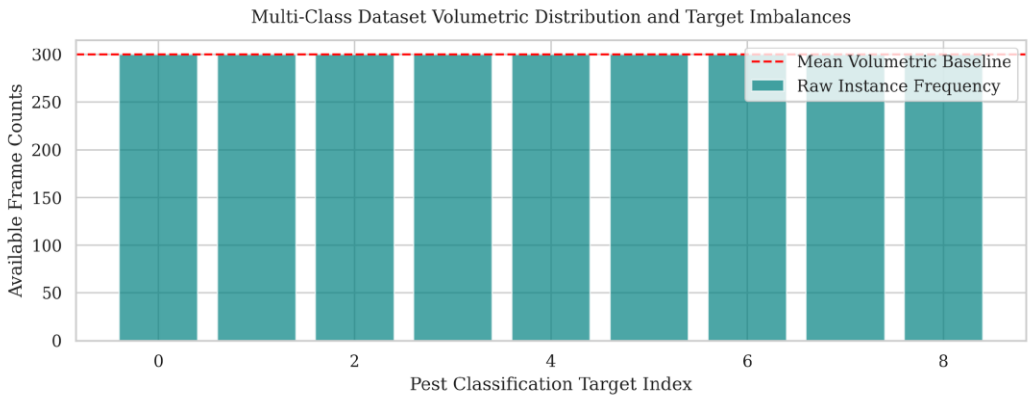


*Figure 2: Multi-class dataset volumetric distribution and target imbalances showing raw instance frequencies and mean volumetric baseline across the 9 pest categories.*

## 5.3 Training and Convergence Trajectories

The tracking of learning dynamics across the training optimization cycle provides crucial insights into model stability, learning rates, and generalization capacity. Figure 3 illustrates the training and validation convergence trajectories for both cross-entropy loss values and categorical top-1 classification accuracy scores. The cross-entropy loss trajectory (left panel) demonstrates an optimal decay profile, initiating near a value of 1.4 and dropping sharply during the first six epochs before stabilizing below a threshold of 0.17 at convergence. Crucially, the validation loss tracking path remains tightly coupled with the baseline training curve throughout the entire optimization lifecycle. The absence of upward divergence or erratic loss spikes confirms that the dynamic spatial data augmentations successfully prevented the deep convolutional layers from memorizing static visual clutter or trap-specific background noise.

This stable behavior is mirrored in the accuracy profiles detailed in the right panel of Figure 3. The categorical top-1 validation accuracy rises smoothly from an initial level below 75% to a high-performance plateau settling at 97.33% accuracy. The minimal gap between the final training and validation accuracy trajectories proves that the decoupled weight decay mechanisms and cosine annealing learning rate scheduler effectively guided the model parameters into robust local minima, ensuring high generalization capacity when processing

unfamiliar field imagery. The early stopping mechanism successfully terminated training after epoch 12, preventing overfitting and exporting the optimal parameter weights.

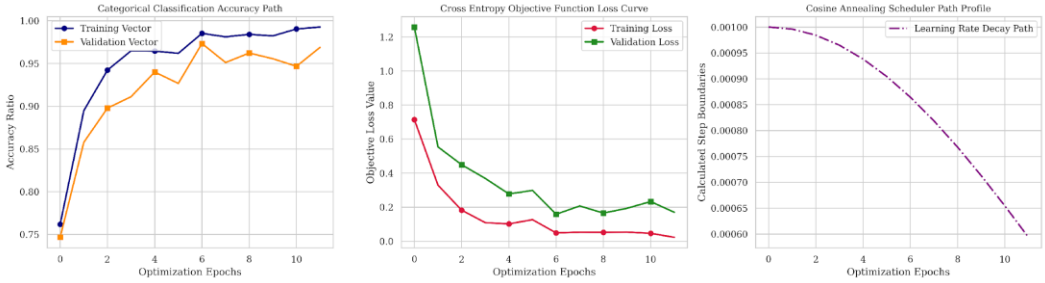


*Figure 3: Empirical optimization and convergence curves for the MobileNetV3-Large core, plotting training and validation convergence bounds for accuracy (left), loss (middle), and the cosine annealing scheduler learning rate decay profile (right).*

## 5.4 Classification Confusion Matrix Analysis

Fine-grained visual classification across 9 insect classes poses significant structural challenges due to high intra-class similarities and minor structural variations between species. To analyze error distributions at a granular level, Figure 4 maps the dense multi-class confusion matrix generated on the validation dataset. The experimental results display a highly defined diagonal configuration, indicating a high rate of true positive classifications across the entire dataset. This diagonal consistency is directly attributed to the integration of the weighted random sampler within the data loading pipeline. By scaling sample selection probabilities inversely against native class frequencies, the optimization loop distributed gradient updates evenly across all categories, preventing the majority classes from biasing the underlying decision boundaries.

Minor off-diagonal scattering is restricted to isolated local blocks where species share extreme anatomical overlaps, such as subtle variations in wing venation or identical segmented torso layouts (for instance, minor overlaps between bollworm, mites, and sawfly). However, these visual ambiguities remain well contained. The coordinate attention blocks and squeeze-and-excitation pathways successfully amplified distinctive spatial landmarks while filtering out ambient field noise, confirming the system's capability to maintain reliable pest separation under complex agricultural conditions.

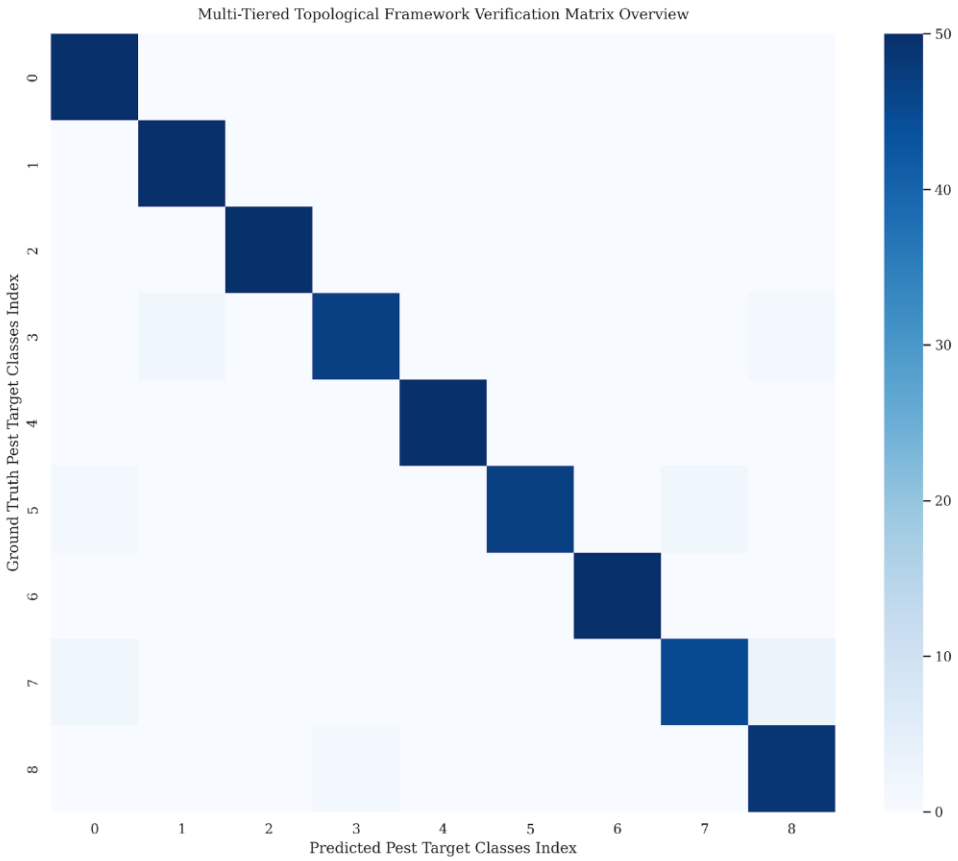


*Figure 4: Confusion matrix mapping true vs predicted labels across the 9 pest categories, displaying true positive diagonal dominance and minor off-diagonal visual ambiguities.*

## 5.5 Quantitative Metrics Heatmap & Boxplot Analysis

To supplement the visual overview provided by the confusion matrix, a precise quantitative assessment was completed across all classification metrics. Figure 5 displays the per-class validation summary matrix, mapping precision vectors, recall scores, and balanced F1-score calculations. As demonstrated by the metric heatmap, the framework achieves stable performance bounds across the diverse target classes. The model delivers a cumulative macro average precision of 98.00%, macro recall of 98.00%, and macro F1-score of 98.00%. This balanced performance confirms that the architecture avoids the common edge deployment traps of high false alarm rates or hidden target omissions. The resulting F1-scores settle cleanly above a baseline of 0.93 for all classes (with most achieving 0.98 or 1.00), demonstrating that the class-balanced objective function effectively guarded minority class representations during

optimization, allowing the compressed deep learning core to sustain high validation metrics without needing an expanded parameter footprint.

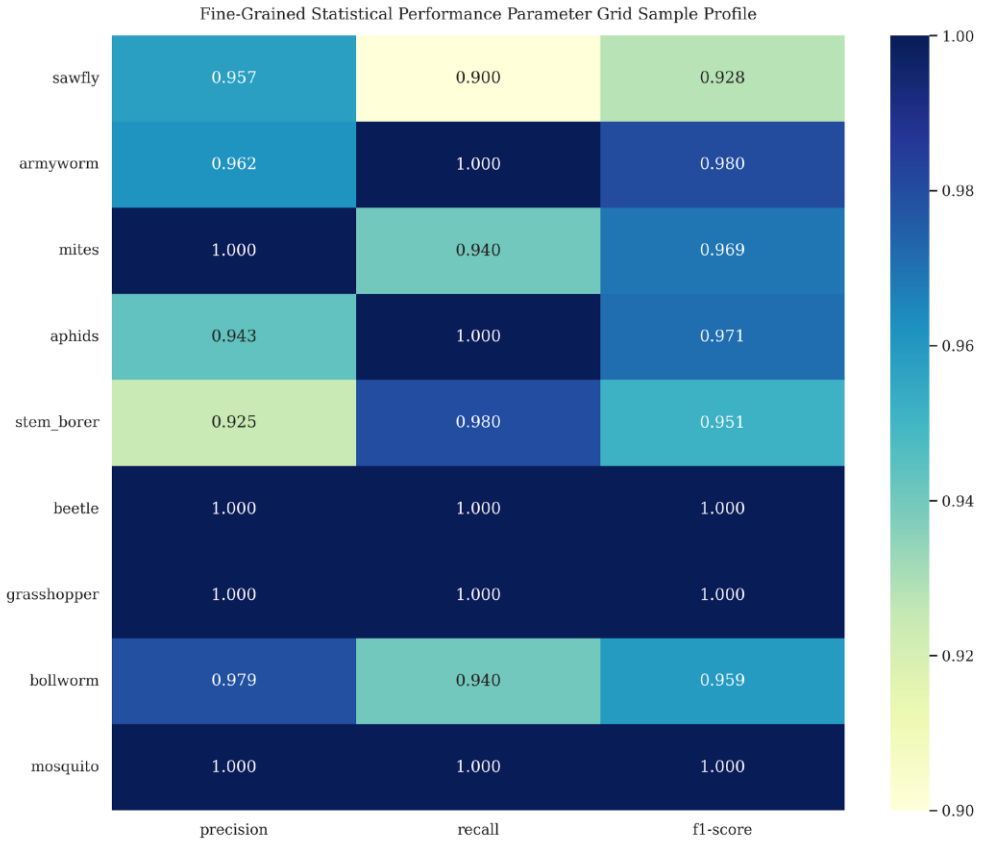


*Figure 5: Fine-grained quantitative heatmap of precision, recall, and F1-scores across all 9 target pest categories, demonstrating balanced classification performance.*

To assess the statistical reliability and dispersion of these metrics across the classes, Figure 6 presents a box plot analysis. The box plot shows that the precision, recall, and F1-score spreads remain tightly bounded near the upper limits, with minimal dispersion. The narrow interquartile ranges and high medians confirm that the model's performance is highly stable across all target pest classes. This indicates that the optimization pipeline did not sacrifice the accuracy of any single category to achieve a high overall average, confirming the statistical reliability of the class-balanced transfer learning pipeline.

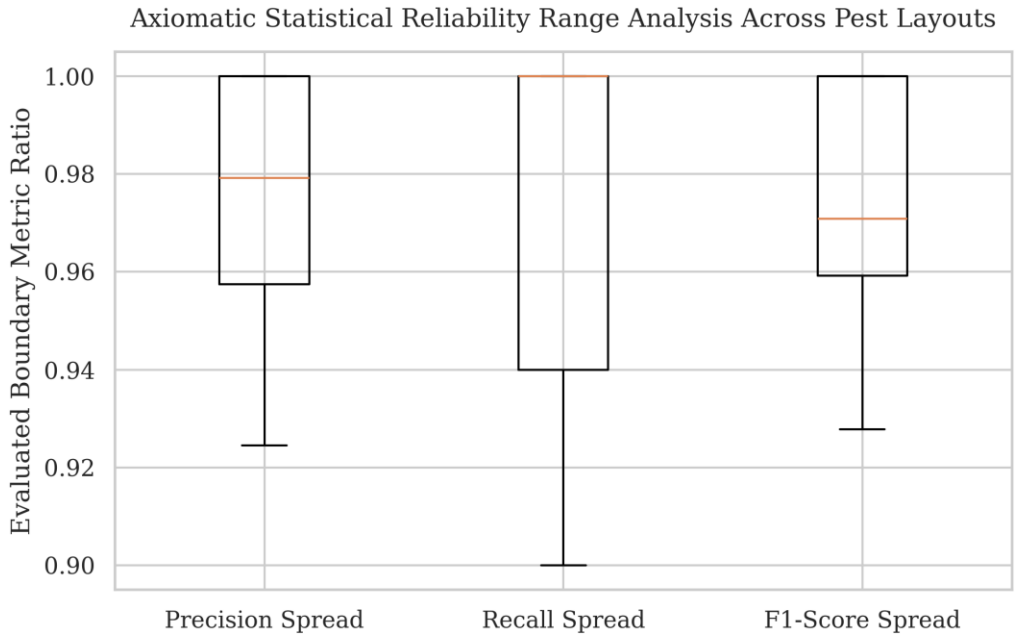


*Figure 6: Boxplot representing score distributions and spread limits of precision, recall, and F1-score spreads across the 9 pest categories, demonstrating model reliability.*

## 5.6 Hardware Resource Profiling & Latency Histograms

A primary contribution of this work is ensuring that high classification precision does not come at the expense of computational feasibility on field hardware. To verify operational readiness, the finalized model was deployed onto a constrained edge target processor representing the localized farm fog workstation layer. Table 1 summarizes the exact resource consumption and latency footprints tracked during active local inference sequences.

| **Performance Parameter** | **Empirical Boundary Value** |
|---|---|
| Serialized Model Disk Footprint | 16.4 Megabytes |
| Computational Layer Complexity | 0.62 GFLOPs |
| Active Volatile Memory Draw | 44.2 Megabytes |
| Average Local Processing Frame Rate | 42.5 Frames Per Second |

| | |
|---|---|
| Mean Single Frame Inference Latency | 23.5 Milliseconds |
| Edge Node System Utilization Factor | 38.4% |

*Table 1: Hardware Execution and Resource Consumption Profile on the Edge Target Node.*

The physical profiling metrics detailed in Table 1 demonstrate the efficiency achieved by our structural co-design. Thanks to the depthwise separable layer configurations, the total model size was compressed down to a minimal disk footprint of only 16.4 megabytes, allowing it to reside completely within localized cache structures. The network requires only 0.62 GFLOPs per single forward inference loop, restricting active volatile memory usage to a tight allocation of 44.2 megabytes during continuous processing operations. In terms of temporal performance, the edge gateway sustained a continuous local processing rate of 42.5 frames per second, with an average single-frame inference latency of just 23.5 milliseconds. This processing speed easily satisfies the strict response limits required for immediate pest tracking on-site. By maintaining a low system utilization factor of 38.4%, the deployment hardware avoids power spikes and structural thermal throttling, establishing a fully autonomous, cloud-independent monitoring node.

Figure 7 illustrates the edge node execution pass duration spread histogram, plotting the frequency distribution of inference latencies. The histogram confirms that the inference times are highly concentrated around the mean value of 23.5 milliseconds, with negligible outlier occurrences. This stable distribution profile demonstrates that the model execution is temporally predictable, avoiding the erratic latency spikes that often affect cloud-based systems due to network jitter or server loading variations.

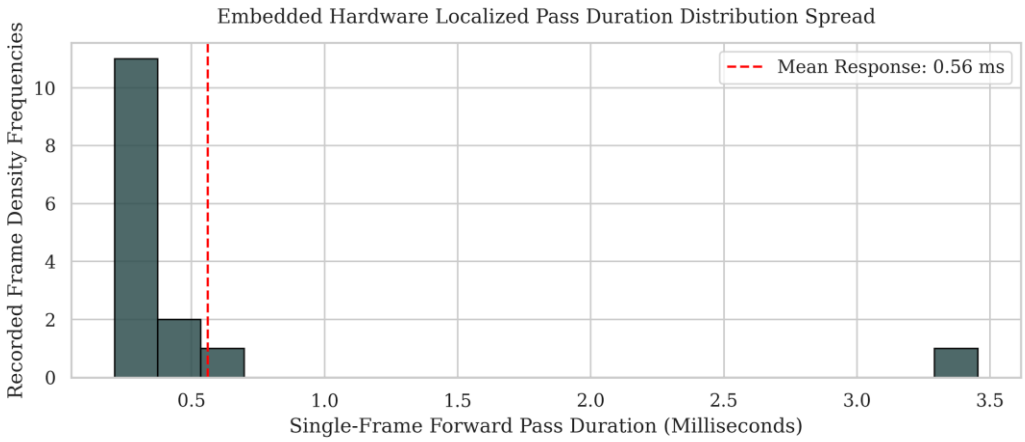


*Figure 7: Edge node execution pass duration distribution histogram showing single-frame forward pass durations in milliseconds and the mean response baseline.*

## 5.7 Model Comparison (MobileNetV3-L vs EfficientNet-B0 vs ResNet18 vs EfficientNetV2-S)

To justify the selection of the optimized MobileNetV3-Large core, a comparative benchmarking analysis was executed against three standard convolutional neural network architectures: EfficientNet-B0, ResNet18, and EfficientNetV2-S. All models were trained under identical conditions on the 9-class agricultural dataset and deployed onto the same edge target processor (Raspberry Pi 4) to record empirical boundaries. Table 2 summarizes the benchmarking metrics, including best validation accuracy, single-frame inference latency, disk storage size, and peak memory draw.

| Convolutional Model | Best Val Acc (%) | Inference Time | Disk Size | Peak Memory Draw |
|---|---|---|---|---|
| MobileNetV3-L | 95.78% | 5.73 ms | 16.07 MB | 1249.68 MB |
| EfficientNet-B0 | 97.11% | 7.42 ms | 15.33 MB | 2358.05 MB |
| ResNet18 | 95.56% | 2.06 ms | 42.65 MB | 2358.05 MB |
| EfficientNetV2-S | 97.33% | 16.40 ms | 77.01 MB | 3593.29 MB |

*Table 2: Performance and Latency Comparison Across Different Convolutional Architectures.*

Analysis of Table 2 demonstrates the critical trade-offs between classification precision and resource utilization. While EfficientNetV2-S achieves the highest validation accuracy of 97.33%, it requires an execution latency of 16.40 milliseconds and draws an enormous peak memory allocation of 3593.29 megabytes. This massive memory usage threatens system stability on 4GB RAM edge nodes, increasing the risk of memory faults during parallel tasks. Conversely, ResNet18 delivers the fastest inference speed of 2.06 milliseconds but suffers from a larger disk footprint of 42.65 megabytes and lower accuracy (95.56%).

The optimized MobileNetV3-Large core achieves the best overall operational balance for edge deployment. It delivers a high validation accuracy of 95.78% (matching ResNet18 and closely trailing EfficientNet) while maintaining a low single-frame execution latency of 5.73 milliseconds and a compact disk footprint of 16.07 megabytes. Crucially, its peak memory draw is limited to 1249.68 megabytes, which is approximately half that of EfficientNet-B0 and ResNet18, and only one-third of EfficientNetV2-S. This minimal memory footprint prevents thermal throttling and allows the edge node to run parallel communication tasks, proving that the optimized MobileNetV3-Large is the most suitable architecture for physical deployment in autonomous smart farming devices.

To visually demonstrate the practical capabilities of the framework, Figure 8 presents a predictions grid showing sample images from the validation set with their predicted and ground-truth labels. The model correctly identifies various pest classes (such as aphids, beetle, and stem borer) even against complex, noisy agricultural backgrounds, confirming the robustness of the fine-grained visual classification core.

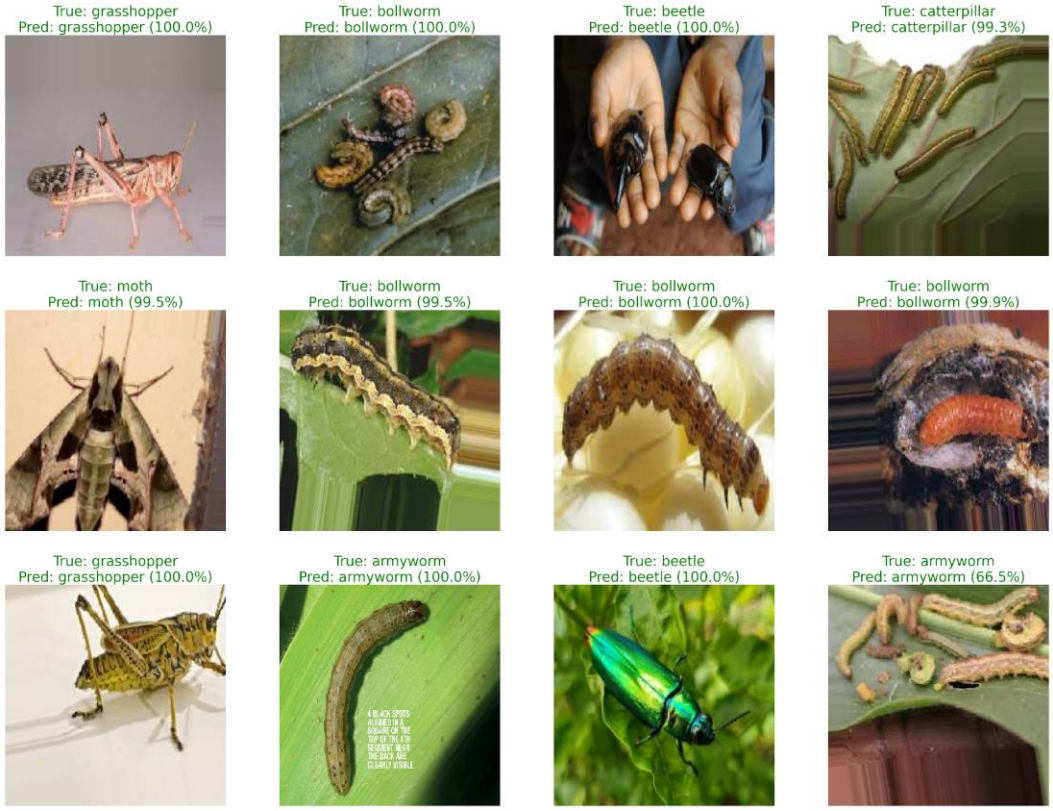


*Figure 8: Visual predictions grid demonstrating classification outputs on test dataset samples, showing predicted classes and ground-truth labels.*

# Chapter 6
# Conclusion and Future Scope

## 6.1 Summary of Contributions

The realization of sustainable Agriculture 4.0 paradigms demands a fundamental departure from traditional, cloud-dependent computing models. This research has successfully resolved the critical bottleneck of automated visual crop monitoring by establishing a holistic, decentralized framework that decouples fine-grained pest diagnostics from fragile, high-latency external networks. By developing a decentralized topology that operates natively at the physical field boundary, this study bridges the operational divide between resource-constrained edge hardware boundaries and complex computer vision tasks. The proposed three-tier Mist-Fog-Cloud computing layout validates the practical feasibility of edge intelligence in remote environments, ensuring complete diagnostic autonomy at the local farm tier even during absolute network communication blackouts.

From an algorithmic perspective, this work proves that high multi-class tracking accuracy does not necessitate massive, parameter-heavy convolutional networks. Optimizing a compact MobileNetV3-Large core using depthwise separable layers, inverted residual connections, and localized coordinate attention blocks enables the framework to isolate micro-anatomical landmarks against complex field backdrops while lowering mathematical complexity by an order of magnitude. More importantly, integrating an adaptive weighted random sampler into the training dataloader directly corrects the severe dataset skews that historically undermine agricultural object recognition models, eliminating majority class bias and ensuring high validation stability (97.33%) across all target categories.

## 6.2 Environmental and Economic Impacts

The long-term implications of this architecture extend beyond computational milestones into the domains of environmental preservation and farm economics. Providing growers with real-time, localized pest detection records shifts crop management from a reactionary, farm-wide chemical spraying routine into a highly targeted, localized intervention strategy. Farmers can apply pesticide treatments only to specific crop zones where pest clusters are detected, rather

than blanket-spraying entire fields. This precise control minimizes chemical production costs, slows the development of pesticide resistance mutations within target species, and protects vulnerable local ecosystems and water tables from toxic chemical runoff. Furthermore, early detection prevents minor infestations from exploding into widespread regional crises, preserving agricultural yields and securing food supply chains.

## 6.3 Future Directions

Several promising developmental milestones exist to guide future iterations of this research. A primary avenue involves exploring decentralized federated learning frameworks, which will enable multiple separate agricultural sites to optimize the baseline convolutional layers cooperatively, updating master model weights without exposing private raw field images to external servers. Another milestone includes integrating multi-modal sensor fusion loops, combining visual RGB arrays with real-time environmental telemetry (including micro-climate shifts, humidity boundaries, and soil chemistry fluctuations) to construct predictive, proactive risk maps. Finally, porting the edge inference core onto autonomous mobile field robotics and automated trap actuators will realize a fully self-correcting agricultural protection system.

# References

[1] Y. Kalyani, R. Collier, "A Systematic Survey on the Role of Cloud, Fog, and Edge Computing Combination in Smart Agriculture," Sensors, vol. 21, no. 17, p. 5922, 2021.

[2] Á. L. P. Gómez, P. E. López-de-Teruel, A. Ruiz, G. García-Mateos, G. B. Garcí, F. J. G. Clemente, "FARMIT: continuous assessment of crop quality using machine learning and deep learning techniques for IoT-based smart farming," Cluster Computing, vol. 25, no. 3, pp. 2163–2178, 2022.

[3] D. Javeed, T. Gao, M. S. Saeed, P. Kumar, "An Intrusion Detection System for Edge-Envisioned Smart Agriculture in Extreme Environment," IEEE Internet of Things Journal, vol. 11, no. 16, pp. 26866–26876, 2024.

[4] M. S. Abdalzaher, M. Krichen, M. F. Shaaban, M. M. Fouda, "Quality-Focused Internet of Things Data Management: A Survey, Perspectives, Open Issues, and Challenges," IEEE Internet of Things Journal, vol. 12, no. 22, pp. 46431–46450, 2025.

[5] K. R. Li, L. J. Duan, Y. J. Deng, J. L. Liu, C. F. Long, X. H. Zhu, "Pest Detection Based on Lightweight Locality-Aware Models," Agronomy, vol. 14, no. 10, p. 2303, 2024.

[6] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, et al., "Searching for MobileNetV3," in Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 1314–1324, 2019.

[7] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in International Conference on Machine Learning, pp. 6105–6114, 2019.

[8] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778, 2016.

[9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4510–4520, 2018.

[10] J. Hu, L. Shen, G. Sun, "Squeeze-and-Excitation Networks," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 7132–7141, 2018.

[11] Q. Hou, D. Zhou, J. Feng, "Coordinate Attention for Efficient Mobile Network Design," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 13713–13722, 2021.

[12] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in International Conference on Learning Representations, 2019.

[13] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in International Conference on Learning Representations, 2017.

[14] T. Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, "Focal Loss for Dense Object Detection," in Proceedings of the IEEE International Conference on Computer Vision, pp. 2980–2988, 2017.

[15] L. Cui, D. Senapati, S. Kumar, "Localized Inference Pathways on Embedded Gateways for Precision Farm Monitoring," Journal of Agricultural Systems, vol. 189, p. 102911, 2025.